Hill Climbing

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

Following are some main features of Hill Climbing Algorithm:

Generate and Test variant: Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.

Greedy approach: Hill-climbing algorithm search moves in the direction which optimizes the cost.

No backtracking: It does not backtrack the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.

Generalized pseudo-code for a GA

```
GA()
  initialize population

  find fitness of population

  while (termination criteria is reached) do
    parent selection
    crossover with probability pc
    mutation with probability pm
    decode and fitness calculation
    survivor selection
    find best
  return best
```

Knapsack Problem by using Genetic Algorithm

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

There are four items. Each item is associated with some weight (W) and value at item (v).

There is a knapsack (k) with limited capacity that can hold atmost 12 kg.

Knapsack Problem by using Genetic Algorithm

STEP -1: Chromosomes Encoding

C_i

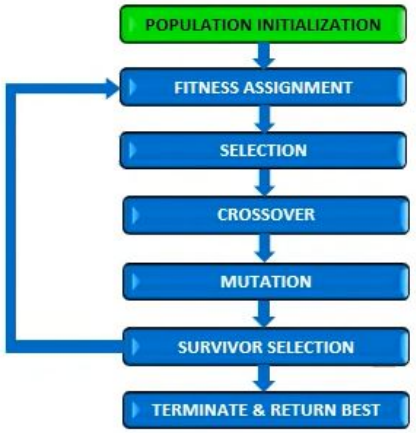
A	B	C	D

Gene: **0 –** represents absence of item in the knapsack
1 – represents presence of item in the knapsack

4 bits are requested to represent chromosomes encoding

Set space = 2^4

Initial population is created and chromosomes randomly created



ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Knapsack Problem by using Genetic Algorithm

STEP -2: Fitness Function

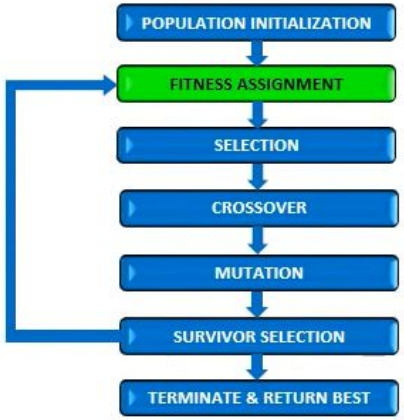
Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_1

C_1	0	1	1	0
	A	B	C	D

represent that knapsack has presence of item B & C and absence of item A & D

- value of knapsack = value of B + value of C
  = 5 + 10 = 15
- Weight of knapsack = weight of item B + weight of item C
  = 3 + 7 = 10 kg

Knapsack capacity = 12 kg as 12kg> 10kg **so C_1 is accepted**

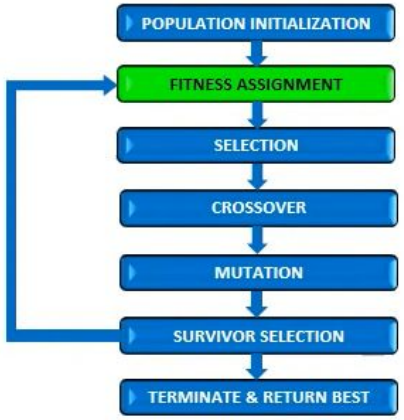


ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -2: Fitness Function

Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_2

C_2	0	1	0	1
	A	B	C	D

represent that knapsack has presence of item B & D and absence of item A & C

- value of knapsack = value of B + value of D
  = 5 + 7 = 12
- Weight of knapsack = weight of item B + weight of item D
  = 3 + 2 = 5 kg

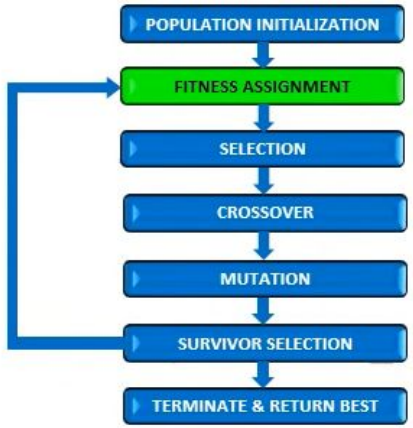
Knapsack capacity = 12 kg as 12kg> 5kg **so C_2 is accepted**

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -2: Fitness Function

Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_3

C_3	1	1	0	1
	A	B	C	D

represent that knapsack has presence of item A, B & D and absence of item C.

- value of knapsack = value of A + value of B + value of D
  = 12 + 5 + 10 = 15
- Weight of knapsack = weight of B + weight of C + weight of D
  = 5 + 3 + 7 = 10 kg

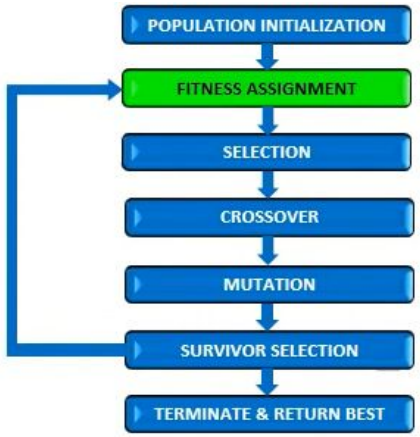
Knapsack capacity = 12 kg as 12kg> 10kg **so C_3 is accepted**

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -2: Fitness Function

Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_4

C_4	1	1	1	1
	A	B	C	D

represent that knapsack has presence of all item A, B, C & D

- value of knapsack = value of A + value of B + value of C + value of D
  = 12 + 5 + 10 + 7 = 34
- Weight of knapsack = weight of B + weight of C + weight of B + weight of C
  = 5 + 3 + 7 + 2 = 17 kg

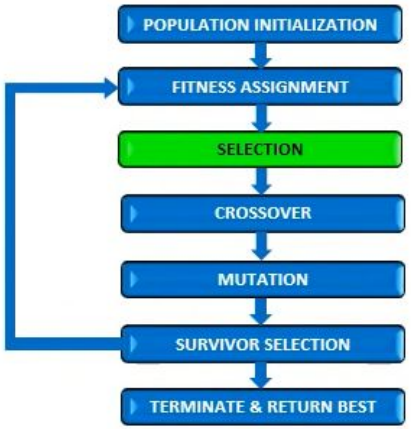
As 17 kg > knapsack capacity; 17 kg > 12 kg **so C_4 is discarded.**

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -3: Selection

Next step is to collect the filter individual and wake up the next generation, chromosome

- **By using Roulette Wheel Selection**

  - Spin the Roulette Wheel and whenever the wheel stops, the individual gets selected at that point.
  - The individual that has the highest fitness value gets larger share of the wheel

e.g. total fitness value = 15 + 12 +24 + 0 = 51

Fitness value of $\mathbf{C_3}$ = 24 ; largest fitness

So, $\mathbf{C_3}$ occupies half of the wheel as 24/51

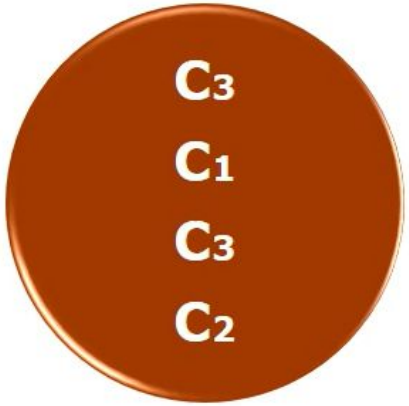
$\mathbf{C_4}$ has zero chance of wining, $\mathbf{C_3}$ has the highest probability of getting selected in the next generation

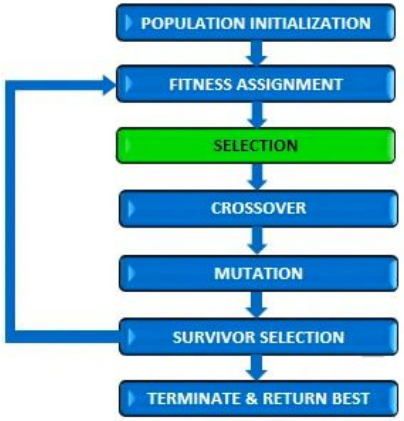
Knapsack Problem by using Genetic Algorithm

After spinning Roulette wheel, in the first spin **C_3** will be selected and then **C_1** after that **C_3** and **C_2**.

Individuals of next generation are selected as follows:

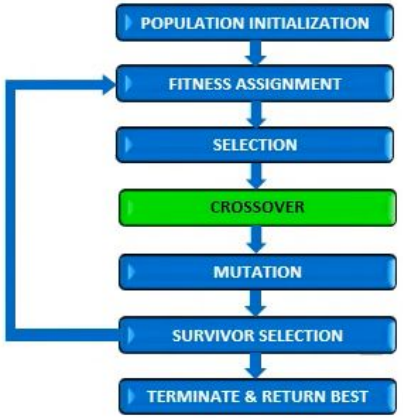
Generation 2





ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Knapsack Problem by using Genetic Algorithm



STEP -4: Crossover

The crossover operation takes the selected chromosomes for mating and mixes the genetic material to produce offspring.

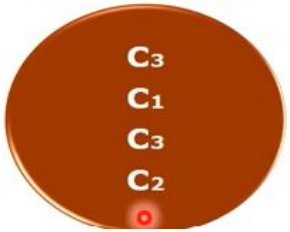
C_3	1	1	0	1
C_2	0	1	1	0
	A	B	C	D

One point crossover – Randomly select the position on the chromosomes about which gene would be exchange.

C_3	1	1	0	1
C_2	0	1	1	0

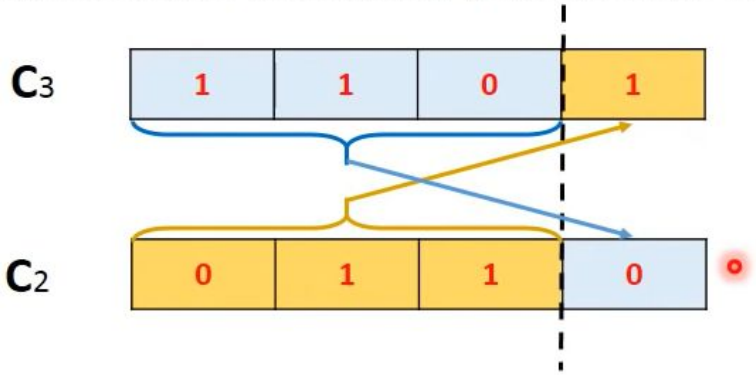
ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Generation 2

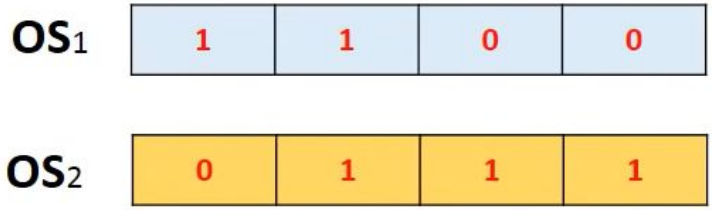


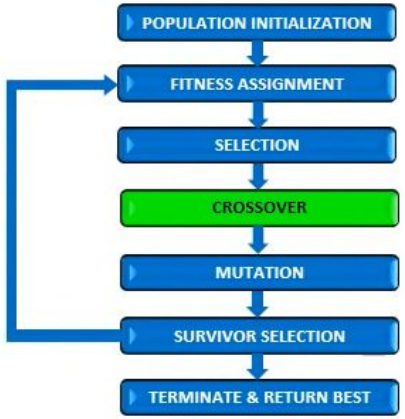
Knapsack Problem by using Genetic Algorithm

One point crossover – Randomly select the position on the chromosomes about which gene would be exchange.



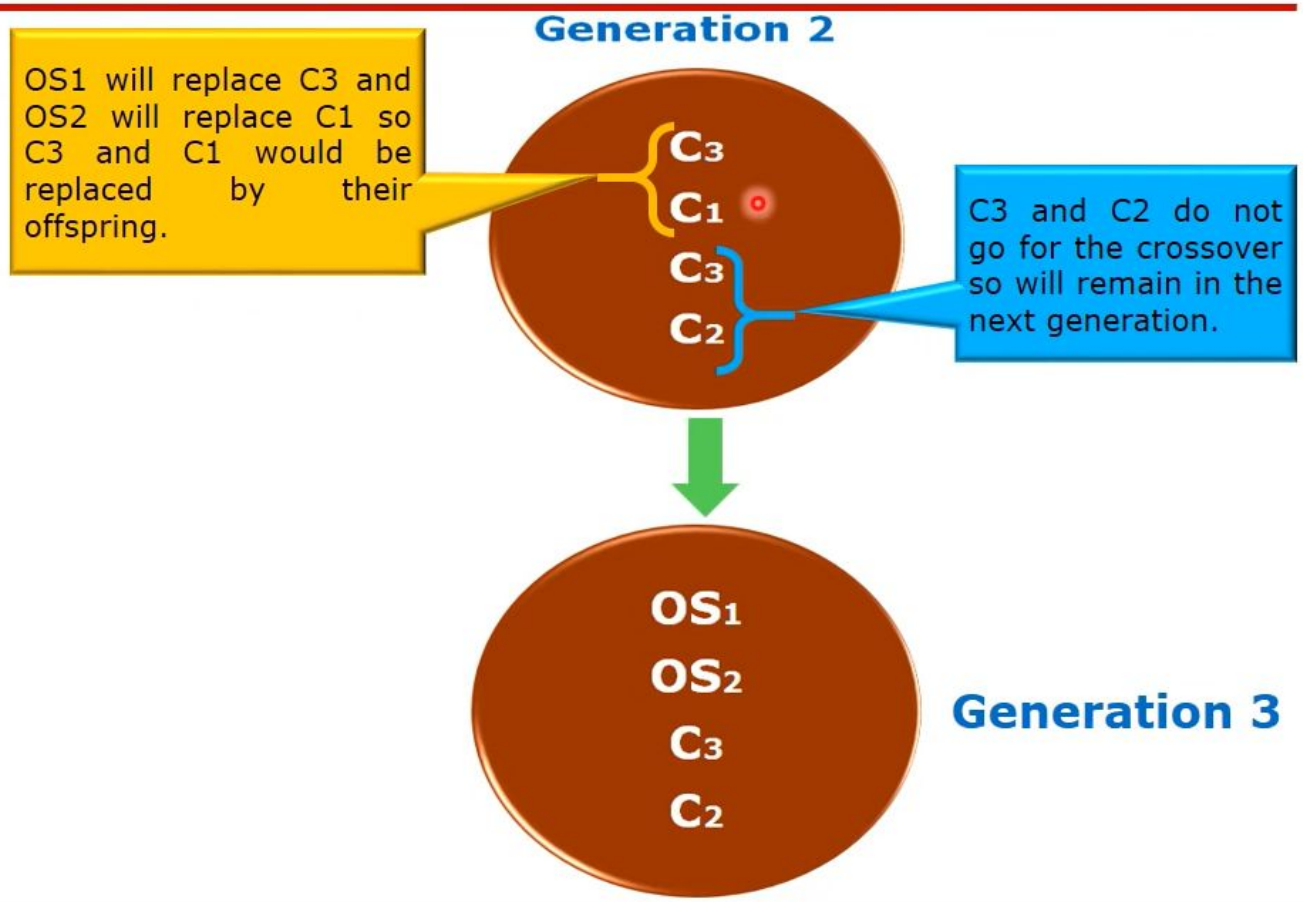
Result of one point crossover i.e. produced offspring

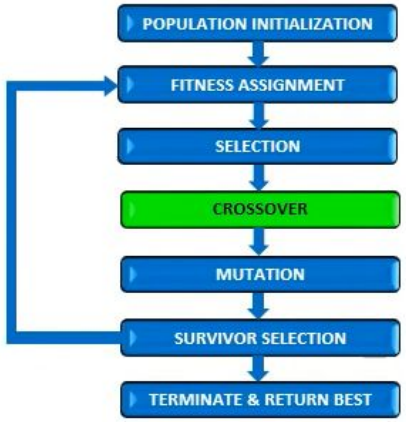




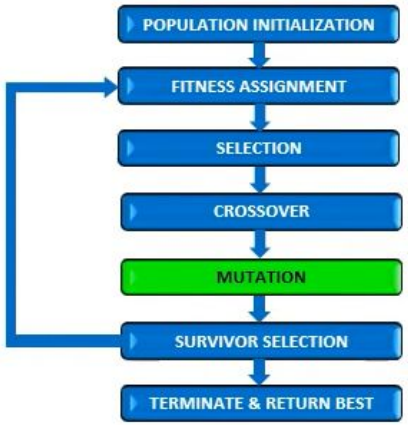
ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Knapsack Problem by using Genetic Algorithm





Knapsack Problem by using Genetic Algorithm

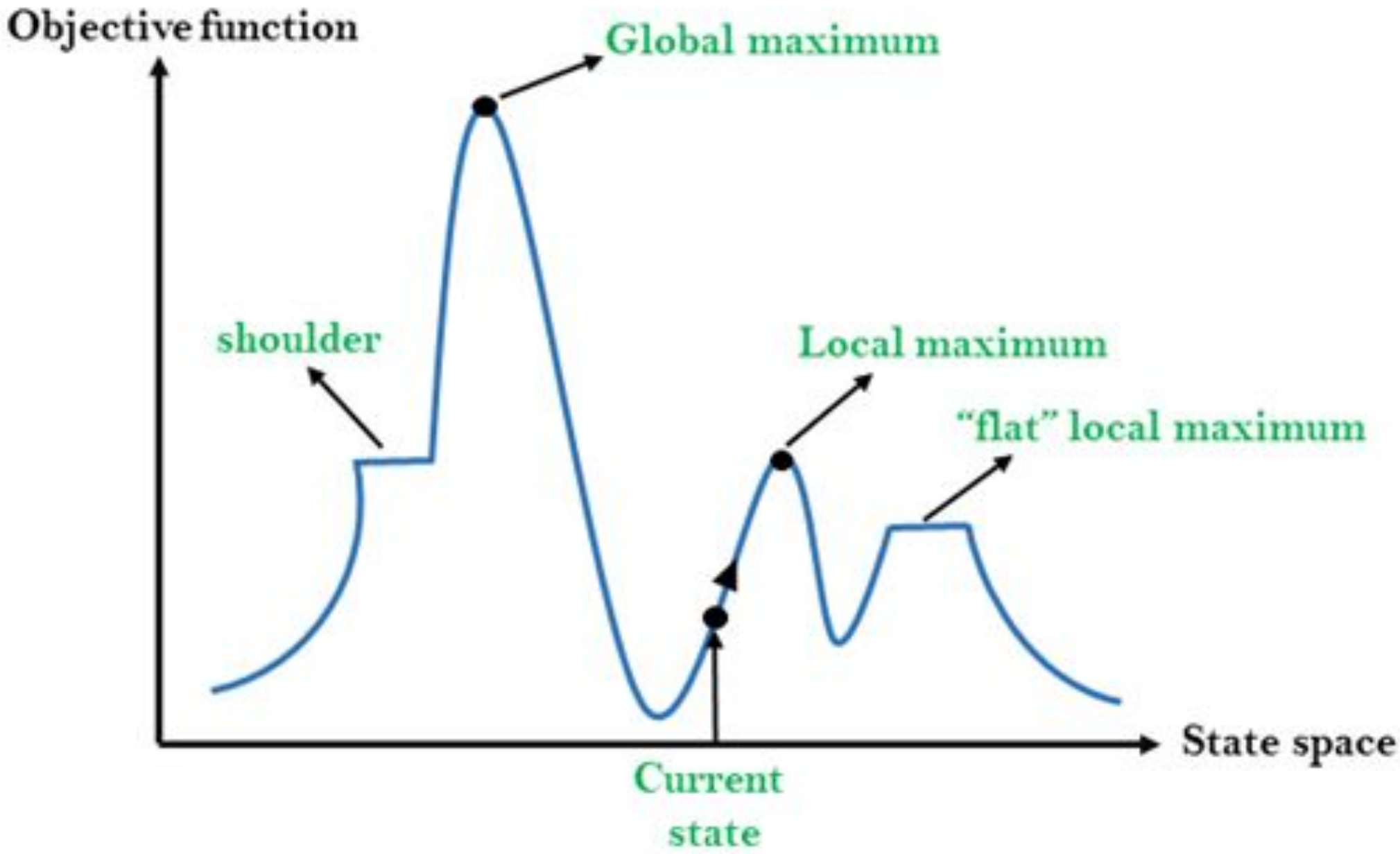


STEP -4: Mutation

Introduces the diversity within the population so that search algorithm does not necessarily get stuck at local maxima.

Lets us consider chromosomes C. Now randomly select a gene from C**.** A Flip happens at the selected genes and zero become one and one becomes zero

C	0	1	1	0
C'	0	1	0	0


Objective function
Global maximum
shoulder
Local maximum
"flat" local maximum
State space
Current
state

Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present.

Flat local maximum: It is a flat space in the landscape where all the neighbor states of current states have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Types of Hill Climbing Algorithm:

Simple hill Climbing:

Steepest-Ascent hill-climbing:

Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. It only evaluates the neighbor node state at a time and s**elects the first one which optimizes current cost and set it as a current state.** It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state. This algorithm has the following features:

Less time consuming

Less optimal solution and the solution is not guaranteed

Algorithm for Simple Hill Climbing:

Step 1: Evaluate the initial state, if it is goal state then return success and Stop.

Step 2: Loop Until a solution is found or there is no new operator left to apply.

Step 3: Select and apply an operator to the current state.

Step 4: Check new state:

If it is goal state, then return success and quit.

Else if it is better than the current state then assign new state as a current state.

Else if not better than the current state, then return to step2.

Step 5: Exit.

Steepest-ascent hill climbing

Steepest-ascent hill climbing is different from simple hill climbing search. Unlike simple hill climbing search, It considers all the successive nodes, compares them, and choose the node which is closest to the solution.

Steepest hill climbing search is similar to **best-first search** because it focuses on each node instead of one.

Steepest-ascent hill climbing algorithm

Create a **CURRENT** node and a **GOAL** node.

If the **CURRENT node=GOAL** node, return **GOAL** and terminate the search.

Loop until a better node is not found to reach the solution.

If there is any better successor node present, expand it.

When the **GOAL** is attained, return **GOAL** and terminate.

The steepest-Ascent algorithm is a variation of simple hill climbing algorithm.

This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm **consumes more time as it searches for multiple neighbors**

Step 1: Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.

Step 2: Loop until a solution is found or the current state does not change.

a-Let SUCC be a state such that any successor of the current state will be better than it.

b- For each operator that applies to the current state:

a-Apply the new operator and generate a new state.
b-Evaluate the new state.
c-If it is goal state, then return it and quit, else compare it to the SUCC.
d-If it is better than SUCC, then set new state as SUCC.
e-If the SUCC is better than the current state, then set current state to SUCC.

Step 3: Exit.

Stochastic hill climbing

Stochastic hill climbing does not focus on all the nodes. It selects one node at random and decides whether it should be expanded or search for a better one.

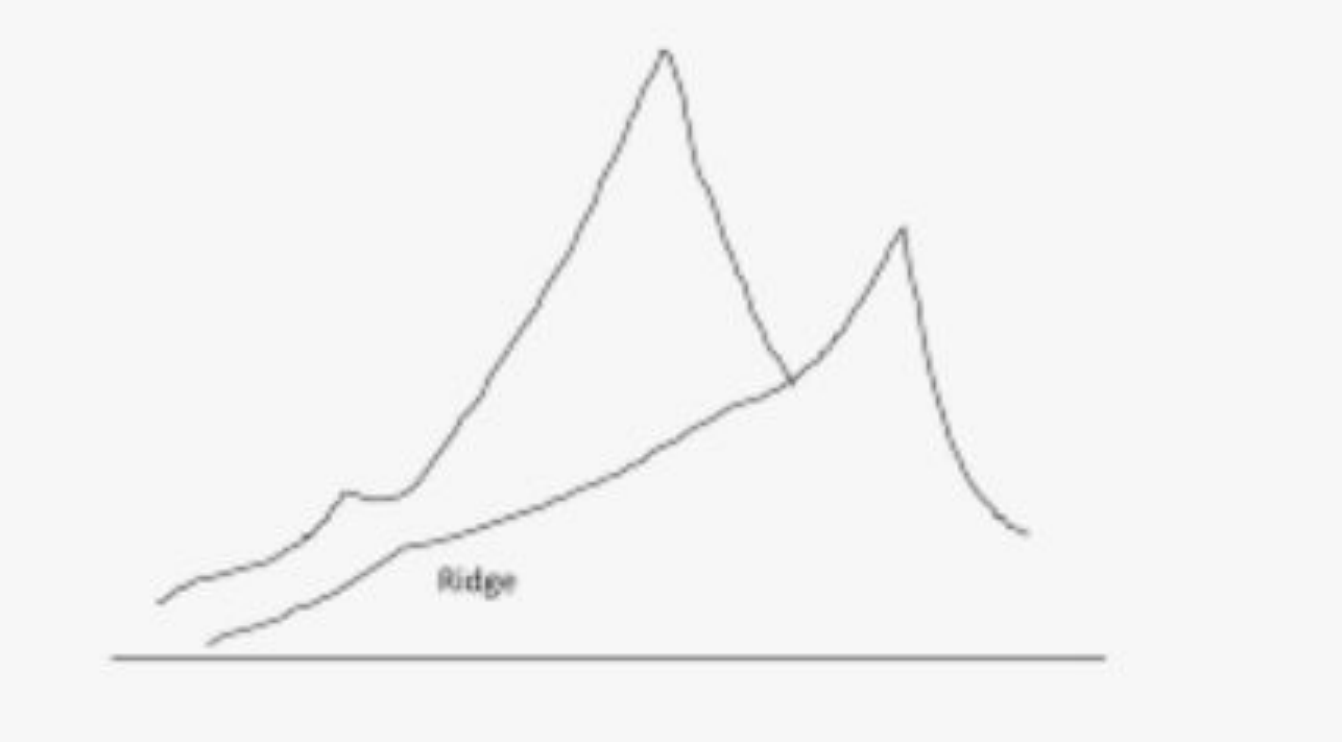
Hill climbing cannot reach the optimal/best state(global maximum) if it enters any of the following regions :

Local maximum: At a local maximum all neighboring states have a value that is worse than the current state. Since hill-climbing uses a greedy approach, it will not move to the worse state and terminate itself. The process will end even though a better solution may exist.
To overcome the local maximum problem: Utilize the backtracking technique. Maintain a list of visited states. If the search reaches an undesirable state, it can backtrack to the previous configuration and explore a new path.

Plateau: On the plateau, all neighbors have the same value. Hence, it is not possible to select the best direction.
To overcome plateaus: Make a big jump. Randomly select a state far away from the current state. Chances are that we will land in a non-plateau region.

Ridge: Any point on a ridge can look like a peak because movement in all possible directions is downward. Hence the algorithm stops when it reaches this state.

To overcome Ridge: In this kind of obstacle, use two or more rules before testing. It implies moving in several directions at once.



8 Puzzle problem using Hill climbing

1	2	4
5		7
3	6	8

Initial state

1	4	7
2	5	8
3	6	

final state

h1 = the number of misplaced tiles.

h1 is an admissible heuristic because it is clear that any tile that is out of place must be moved at least once.

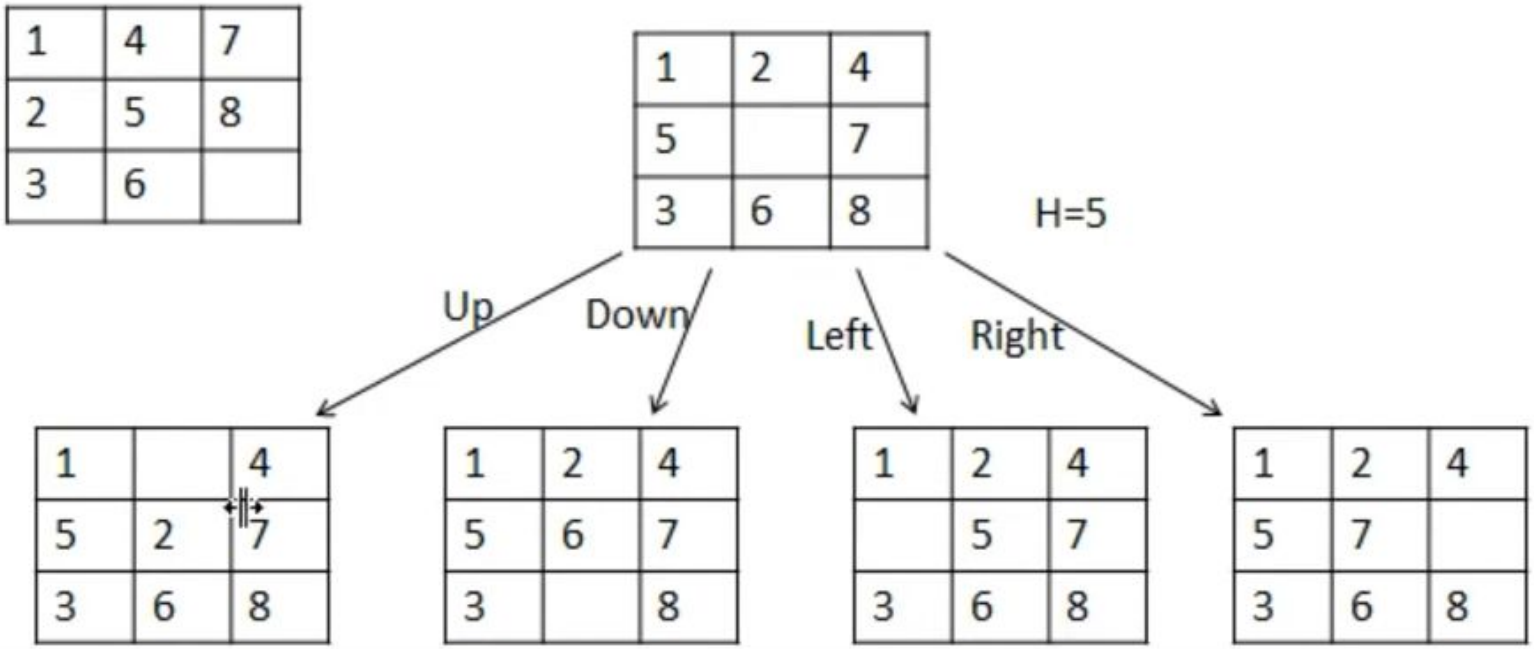
1	**2**	**4**
5		**7**
3	6	**8**

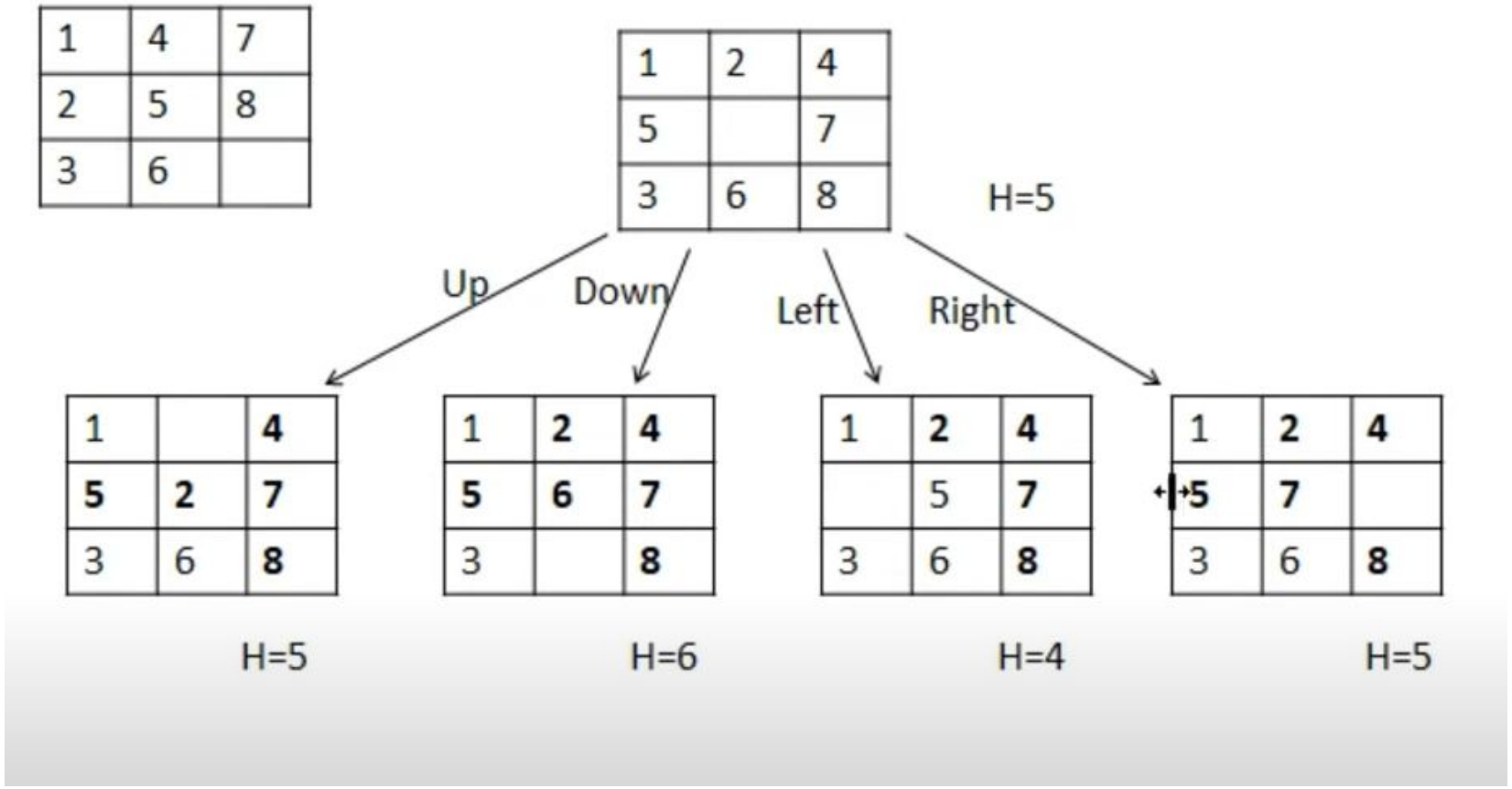
Initial state

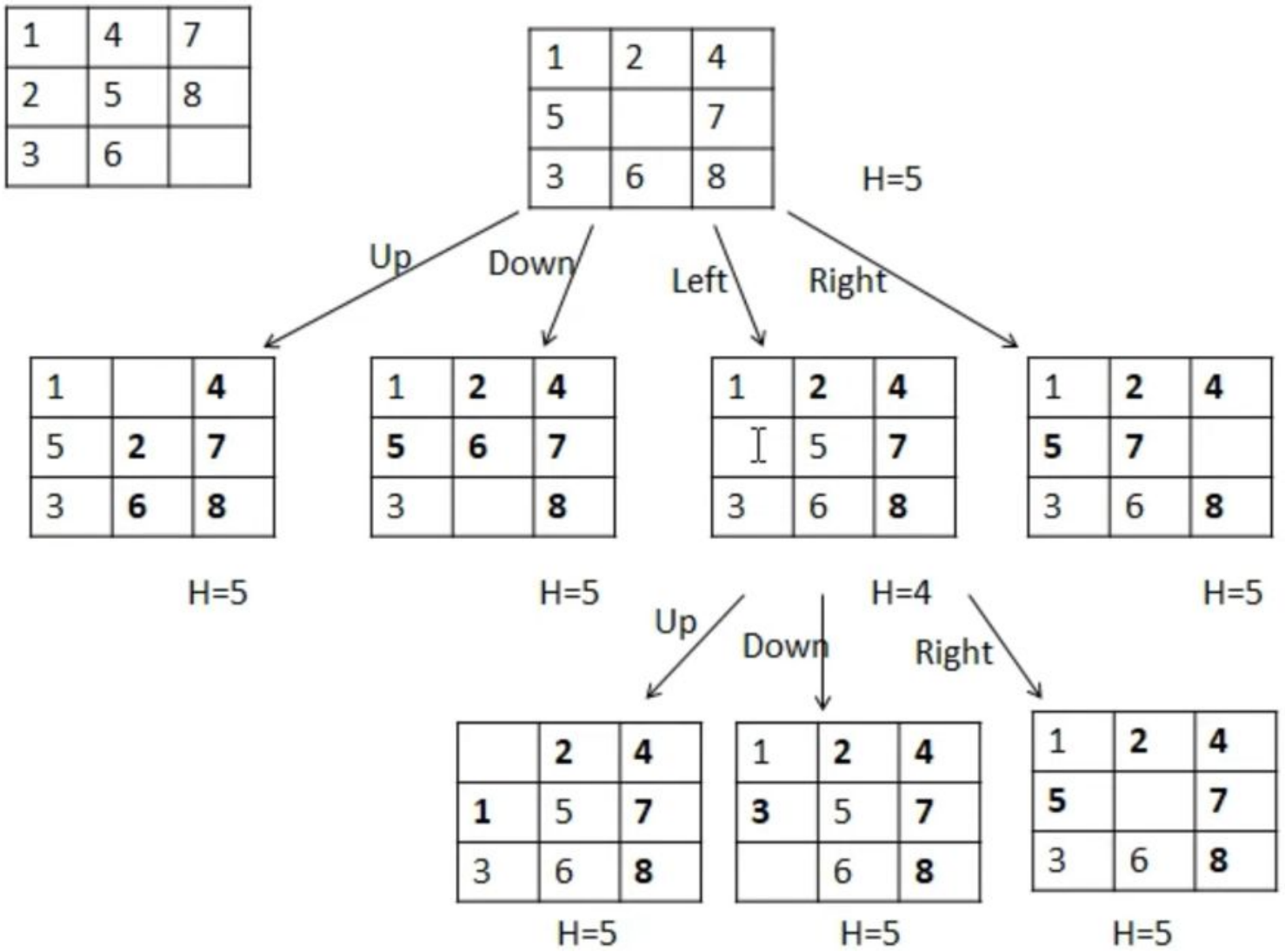
1	4	7
2	5	8
3	6	

final state

Heuristic value =5


1 4 7
2 5 8
3 6
1 2 4
5 7
3 6 8
H=5
Up
Down
Left
Right
1 4
5 2 7
3 6 8
1 2 4
5 6 7
3 8
1 2 4
5 7
3 6 8
1 2 4
5 7
3 6 8


1 4 7
2 5 8
3 6
1 2 4
5 7
3 6 8
H=5
Up
Down
Left
Right
1 4
5 2 7
3 6 8
H=5
1 2 4
5 6 7
3 8
H=6
1 2 4
5 7
3 6 8
H=4
1 2 4
5 7
3 6 8
H=5


1 4 7
2 5 8
3 6
1 2 4
5 7
3 6 8
H=5
Up
Down
Left
Right
1 4
5 2 7
3 6 8
H=5
1 2 4
5 6 7
3 8
H=5
1 2 4
5 7
3 6 8
H=4
1 2 4
5 7
3 6 8
H=5
Up
Down
Right
2 4
1 5 7
3 6 8
H=5
1 2 4
3 5 7
6 8
H=5
1 2 4
5 7
3 6 8
H=5

Simulated Annealing

- A hill-climbing algorithm that *never* makes “downhill” moves toward states with lower value (or higher cost) is guaranteed to be incomplete, because it can get stuck on a local maximum.
- In contrast, a purely random walk—that is, moving to a successor chosen uniformly at random from the set of successors—is complete but extremely inefficient.
- Therefore, it seems reasonable to try to combine hill climbing with a random walk in some way that yields both efficiency and completeness.
- **Simulated annealing** is such an algorithm.
- Simulated annealing was first used extensively to solve VLSI layout problems in the early 1980s. It has been applied widely to factory scheduling and other large-scale optimization tasks.

Simulated Annealing (contd…)

- In metallurgy, **annealing** is the process used to temper or harden metals and glass by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach a low energy crystalline state.
- To explain simulated annealing, we switch our point of view from hill climbing to **gradient descent** (i.e., minimizing cost) and imagine the task of getting a ping-pong ball into the deepest crevice in a bumpy surface.
- If we just let the ball roll, it will come to rest at a local minimum. If we shake the surface, we can bounce the ball out of the local minimum. The trick is to shake just hard enough to bounce the ball out of local minima but not hard enough to dislodge it from the global minimum.
- The simulated-annealing solution is to start by shaking hard (i.e., at a high temperature) and then gradually reduce the intensity of the shaking (i.e., lower the temperature).

Simulated Annealing Algorithm

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
inputs: problem, a problem
        schedule , a mapping from time to "temperature"
current ← MAKE-NODE (problem.INITIAL-STATE)
    for t = 1 to ∞ do
    T ← schedule (t)
    if T = 0 then return current
    Next ← a randomly selected successor of current
ΔE ← next.VALUE – current.VALUE
    if ΔE >0 then current ← next
    else current ← next only with probability e^(ΔE/T)
```

Simulated Annealing Algorithm (contd…)

SIMULATED ANNEALING

The algorithm for simulated annealing is as follows:

1. Let *IS* be the initial state and *GS* be the goal state.
2. Check if *IS* = *GS*. If true, return success.
   Else
   (i) Initialise t for *IS*, where t is temperature or energy level to allow maximum movement
   (ii) Do while a stop condition is not satisfied.
   (a) Randomly pick a neighbour, say x.
   (b) Evaluate x.
   (c) ΔE = Energy_val(x) – Energy_val(*IS*)
   (d) If x = *GS*, return success.
   (e) If $\Delta E < 0$, make x to be *IS*. (This means new state/solution is a better one)
   (f) Else generate random number: $r[0, 1]$.
   If $r < e^{-\Delta E/t}$, make x to be *IS*.
   (g) Revise t values.

(The **simulated annealing algorithm**, a version of stochastic hill climbing where some downhill moves are allowed. Downhill moves are accepted readily early in the annealing schedule and then less often as time goes on. The schedule input determines the value of the temperature T as a function of time.)

Beam Search:

- Beam search uses breadth-first search to build its search tree. At each level of the tree, **it generates all successors of the states at the current level, sorting them in increasing order of heuristic cost**. However, it only stores a **predetermined number (β), of best states at each level called the beamwidth**. Only those states are expanded next.

- The beam width can either be fixed or variable. One approach that uses a variable beam width starts with the width at a minimum. If no solution is found, the beam is widened, and the procedure is repeated.

Uses of Beam Search

- A beam search is most often used to maintain tractability in large systems with insufficient memory to store the entire search tree. For example,It has been used in many machine translation systems.

- Each part is processed to select the best translation, and many different ways of translating the words appear.

- According to their sentence structures, the top best translations are kept, and the rest are discarded. The translator then evaluates the translations according to a given criterion, choosing the translation which best keeps the goals.

- The first use of a beam search was in the Harpy Speech Recognition System, CMU 1976.

Drawbacks of Beam Search

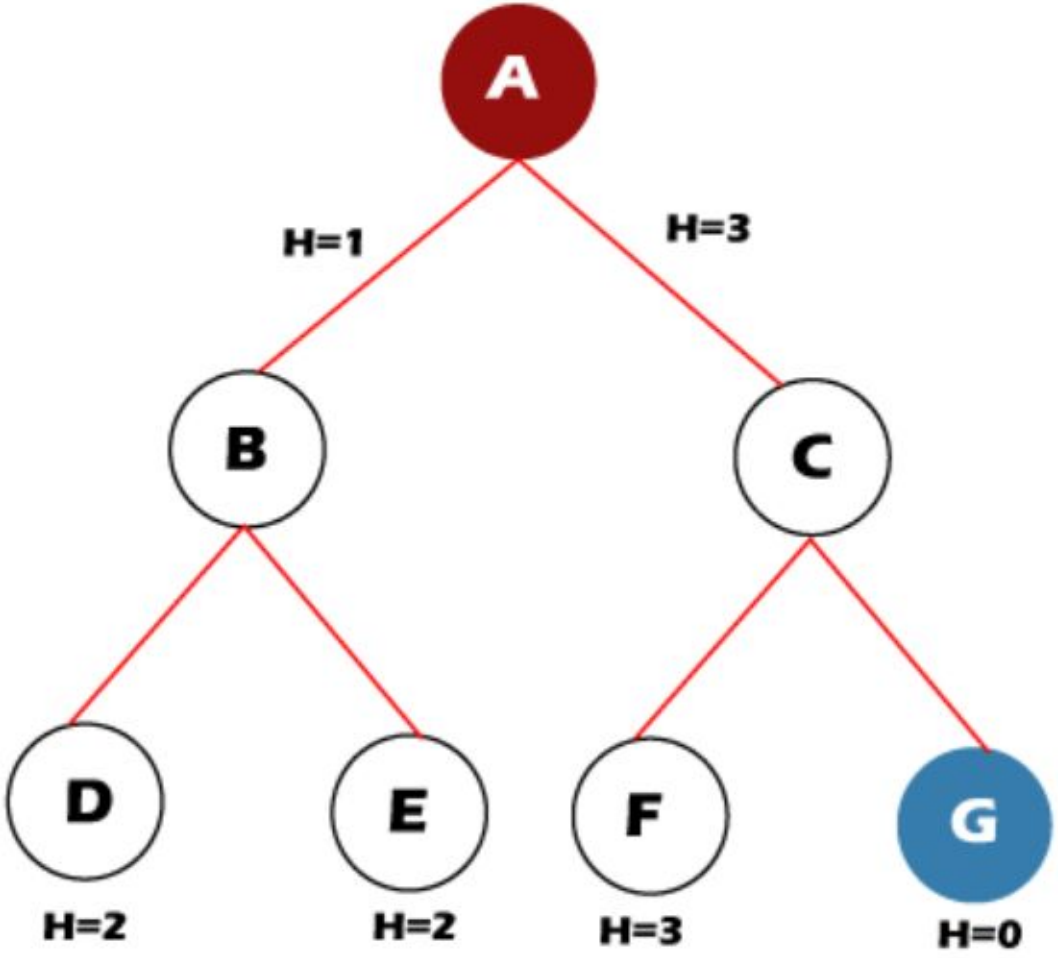
- In general, the Beam Search Algorithm is not complete. Despite these disadvantages, beam search has found success in the practical areas of speech recognition, vision, planning, and machine learning.

- The main disadvantages of a beam search are that the search may not result in an optimal goal and may not even reach a goal at all after given unlimited time and memory when there is a path from the start node to the goal node.

- The beam search algorithm terminates for two cases: a required goal node is reached, or a goal node is not reached, and there are no nodes left to be explored.

- A more accurate heuristic function and a larger beam width can improve Beam Search's chances of finding the goal.

For example, let's take the value of ß = 2 for the tree shown below. So, follow the following steps to find the goal node.



Define Beam Search

Step 1: OPEN= {A}

Step 2: OPEN= {B, C}

Step 3: OPEN= {D, E}

Step 4: OPEN= {E}

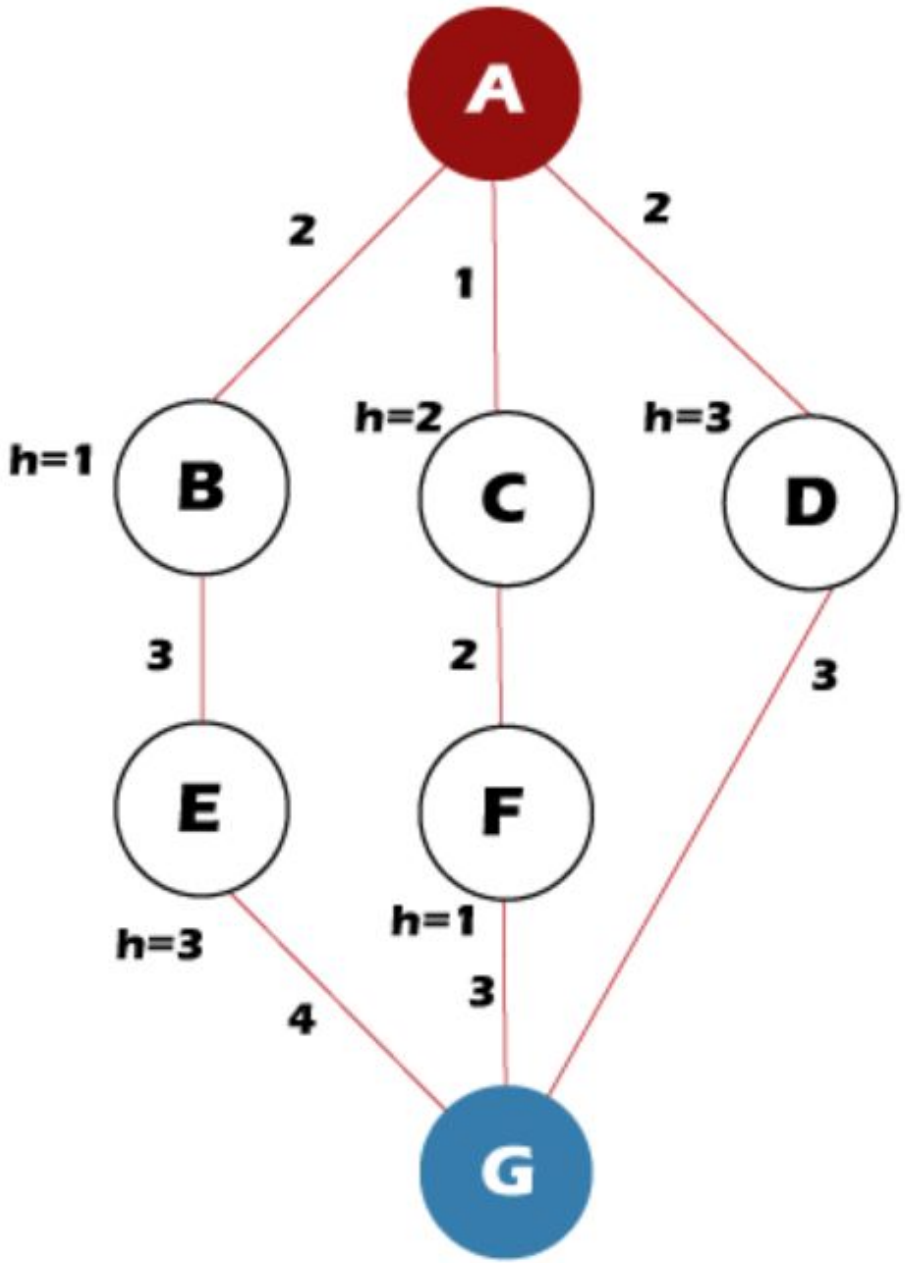
Step 5: OPEN= { }

Beam Search Optimality

The Beam Search algorithm is not complete in some cases. Therefore it is also not guaranteed to be optimal. It can happen because of these reasons:

The beam width and an inaccurate heuristic function may cause the algorithm to miss expanding the shortest path.
A more precise heuristic function and a larger beam width can make Beam Search more likely to find the optimal path to the goal.

Consider the following Graph:



- Follow the following steps to find the path for the goal node.

- Step 1: OPEN= {A}
- Step 2: OPEN= {B, C}
- Step 3: OPEN= {C, E}
- Step 4: OPEN= {F, E}
- Step 5: OPEN= {G, E}
- Step 6: Found the goal node {G}, now stop.

- **Path: A-> C-> F-> G**

- **But the Optimal Path is A-> D-> G**

• Time Complexity of Beam Search

- The time complexity of the Beam Search algorithm depends on the following things, such as:
- The accuracy of the heuristic function.
- In the worst case, the heuristic function leads Beam Search to the deepest level in the search tree.
- The worst-case time = **O(B*m)**
- **B is the beam width, and m is the maximum depth of any path in the search tree.**

• Space Complexity of Beam Search

- The space complexity of the Beam Search algorithm depends on the following things, such as:
- Beam Search's memory consumption is its most desirable trait.
- Since the algorithm only stores B nodes at each level in the search tree.
- The worst-case space complexity = **O(B*m)**
- B is the beam width, and m is the maximum depth of any path in the search tree.

Genetic Algorithm (GA)

GA is a search-based optimization technique based on the principles of Genetics and Natural Selection.

It is frequently used to find optimal or near-optimal solutions to difficult problems which otherwise would take a lifetime to solve. It is frequently used to solve optimization problems, in research, and in machine learning.

Introduction to Optimization

Optimization is the process of making something better. In any process, we have a set of inputs and a set of outputs as shown in the following figure.

Optimization refers to finding the values of inputs in such a way that we get the “best” output values.

The definition of “best” varies from problem to problem, but in mathematical terms, it refers to maximizing or minimizing one or more objective functions, by varying the input parameters.

What are Genetic Algorithms?

Nature has always been a great source of inspiration to all mankind. **Genetic Algorithms** (GAs) are search based algorithms based on the concepts of natural selection and genetics.

GAs are a subset of a much larger branch of computation known as Evolutionary Computation.

GAs were developed by **John Holland** and his students and colleagues at the University of **Michigan**, most notably **David E. Goldberg** and has since been tried on various optimization problems with a high degree of success.

In GAs, we have a pool or a population of possible solutions to the given problem. These solutions then undergo recombination and mutation (like in natural genetics), producing new children, and the process is repeated over various generations.

Each individual (or candidate solution) is assigned a fitness value (based on its objective function value) and the fitter individuals are given a higher chance to mate and yield more "fitter" individuals. This is in line with the **Darwinian Theory of "Survival of the Fittest".**

In this way we keep "evolving" better individuals or solutions over generations, till we reach a stopping criterion.

Genetic Algorithms are sufficiently randomized in nature, but they perform much better than random local search (in which we just try various random solutions, keeping track of the best so far), as they exploit historical information as well.

Advantages of GAs

GAs have various advantages which have made them immensely popular. These include −

Does not require any derivative information (which may not be available for many real-world problems).

Is faster and more efficient as compared to the traditional methods.

Has very good parallel capabilities.

Optimizes both continuous and discrete functions and also multi-objective problems.

Provides a list of “good” solutions and not just a single solution.

Always gets an answer to the problem, which gets better over the time.

Useful when the search space is very large and there are a large number of parameters involved.

Limitations of GAs

Like any technique, GAs also suffer from a few limitations. These include −

GAs are not suited for all problems, especially problems which are simple and for which derivative information is available.

Fitness value is calculated repeatedly which might be computationally expensive for some problems.

Being stochastic, there are no guarantees on the optimality or the quality of the solution.

If not implemented properly, the GA may not converge to the optimal solution.

GA – Motivation

Genetic Algorithms have the ability to deliver a “good-enough” solution “fast-enough”. This makes genetic algorithms attractive for use in solving optimization problems. The reasons why GAs are needed are as follows −

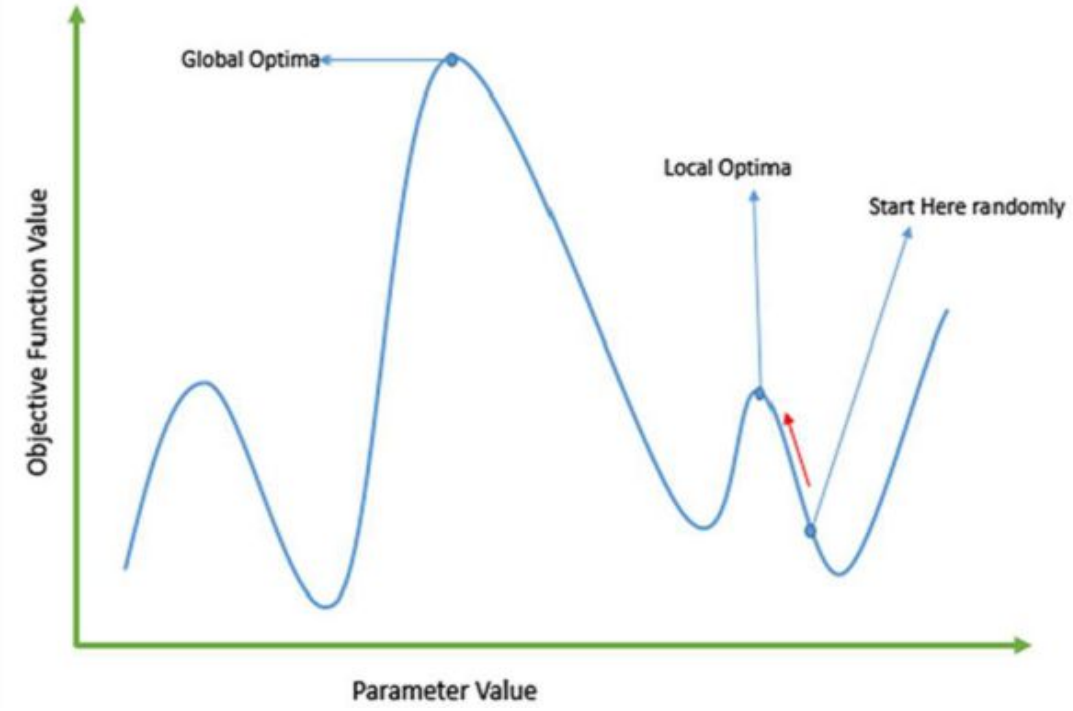
Solving Difficult Problems

In computer science, there is a large set of problems, which are **NP-Hard**. What this essentially means is that, even the most powerful computing systems take a very long time (even years!) to solve that problem.

In such a scenario, GAs prove to be an efficient tool to provide usable near-optimal solutions in a short amount of time.

Failure of Gradient Based Methods

Traditional calculus based methods work by **starting at a random point** and by moving in the direction of the gradient, till we reach the top of the hill. This technique is efficient and works very well for **single-peaked objective functions like the cost function in linear regression**. But, in most real-world situations, we have a very complex problem called as landscapes, which are made of many peaks and many valleys, which causes such methods to fail, as they suffer from an inherent tendency of getting stuck at the **local optima** as shown in the following figure.



Getting a Good Solution Fast

Some difficult problems like the **Travelling Salesperson Problem (TSP)**, have real-world applications like path finding and **VLSI Design**.

Now imagine that you are using your **GPS Navigation system**, and it takes a few minutes (or even a few hours) to compute the **"optimal"** path from the source to destination.

Delay in such real world applications is not acceptable and therefore a **"good-enough"** solution, which is delivered "fast" is what is required.

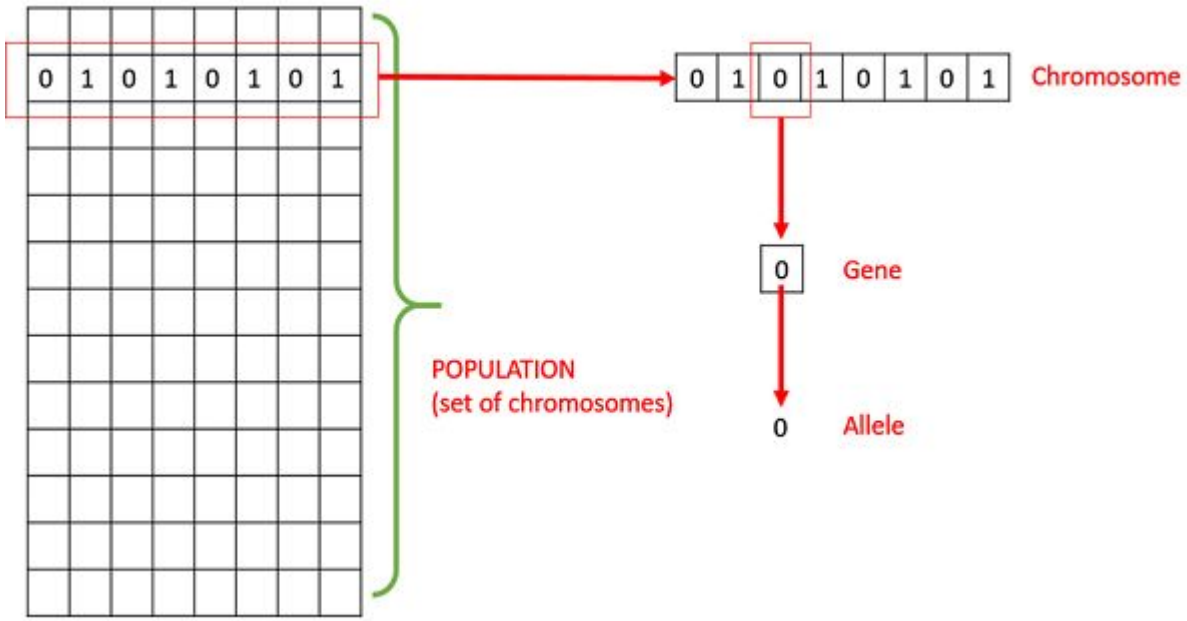
Genetic Algorithms - Fundamentals

Population – It is a subset of all the possible (encoded) solutions to the given problem. The population for a GA is analogous to the population for human beings except that instead of human beings, we have Candidate Solutions representing human beings.

Chromosomes – A chromosome is one such solution to the given problem.

Gene – A gene is one element position of a chromosome.

Allele – It is the value a gene takes for a particular chromosome.



Difference between Genetic Algorithms and Traditional Algorithms

- A search space is the set of all possible solutions to the problem. In the traditional algorithm, only one set of solutions is maintained, whereas, in a genetic algorithm, several sets of solutions in search space can be used.
- Traditional algorithms need more information in order to perform a search, whereas genetic algorithms need only one objective function to calculate the fitness of an individual.
- Traditional Algorithms cannot work parallelly, whereas genetic Algorithms can work parallelly (calculating the fitness of the individualities are independent).
- One big difference in genetic Algorithms is that rather of operating directly on seeker results, inheritable algorithms operate on their representations (or rendering), frequently appertained to as chromosomes.
- Traditional Algorithms can only generate one result in the end, whereas Genetic Algorithms can generate multiple optimal results from different generations.
- Traditional algorithms are deterministic in nature, whereas Genetic algorithms are probabilistic and stochastic in nature.

Population: Population is the subset of all possible or probable solutions, which can solve the given problem.

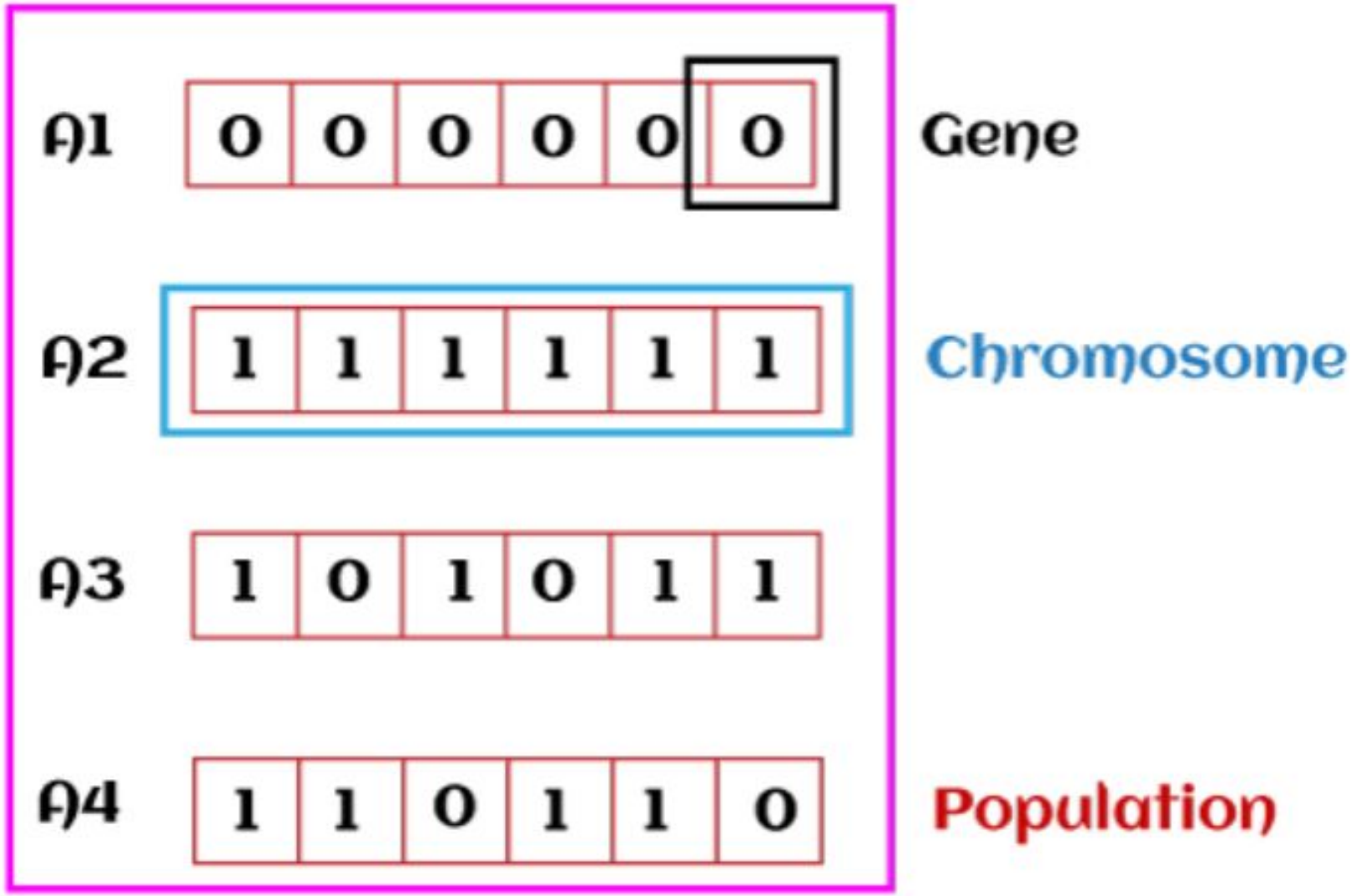
Chromosomes: A chromosome is one of the solutions in the population for the given problem, and the collection of gene generate a chromosome.

Gene: A chromosome is divided into a different gene, or it is an element of the chromosome.

Allele: Allele is the value provided to the gene within a particular chromosome.

Fitness Function: The fitness function is used to determine the individual's fitness level in the population. It means the ability of an individual to compete with other individuals. In every iteration, individuals are evaluated based on their fitness function.

Genetic Operators: In a genetic algorithm, the best individual mate to regenerate offspring better than parents. Here genetic operators play a role in changing the genetic composition of the next generation.


A1
0 0 0 0 0 0
Gene
A2
1 1 1 1 1 1
Chromosome
A3
1 0 1 0 1 1
A4
1 1 0 1 1 0
Population

Genotype – Genotype is the population in the computation space. In the computation space, the solutions are represented in a way which can be easily understood and manipulated using a computing system.

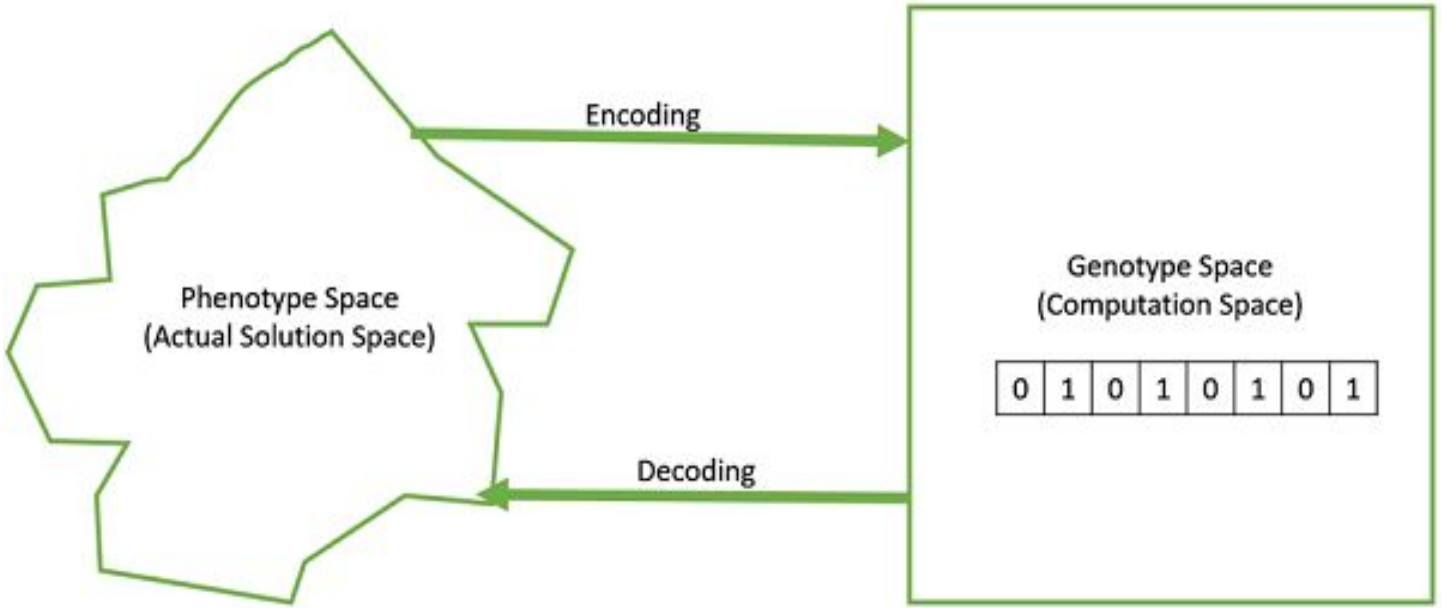
Phenotype – Phenotype is the population in the actual real world solution space in which solutions are represented in a way they are represented in real world situations.

Decoding and Encoding – For simple problems, the phenotype and genotype spaces are the same. However, in most of the cases, the phenotype and genotype spaces are different.

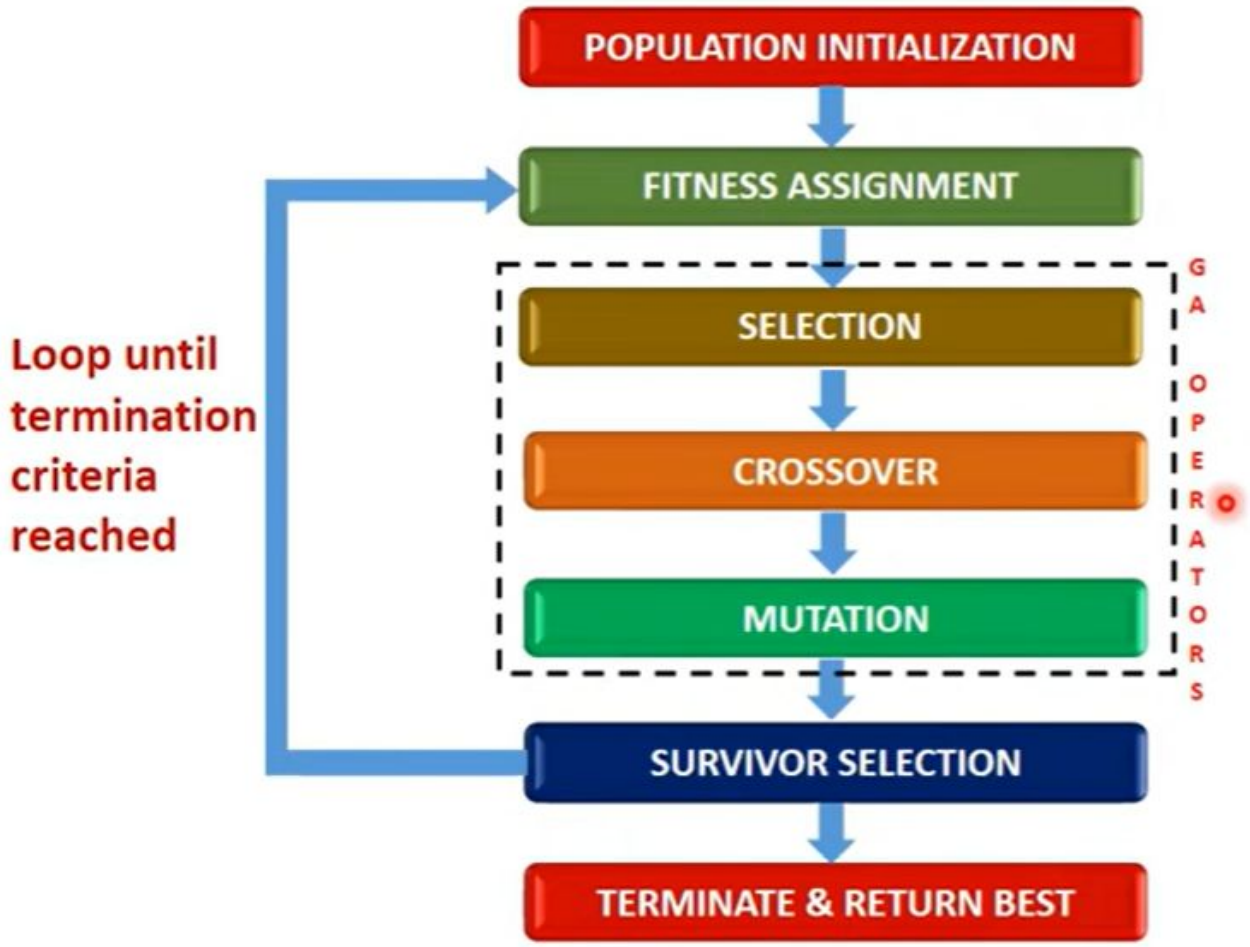
Decoding is a process of transforming a solution from the genotype to the phenotype space, while encoding is a process of transforming from the phenotype to genotype space. Decoding should be fast as it is carried out repeatedly in a GA during the fitness value calculation.

For example, consider the **0/1 Knapsack Problem**. The Phenotype space consists of solutions which just contain the item numbers of the items to be picked.

However, in the genotype space it can be represented as a binary string of length n (where n is the number of items). A 0 at position x represents that xth item is picked while a 1 represents the reverse. This is a case where genotype and phenotype spaces are different.



Structure of a GA



Generalized pseudo-code for a GA

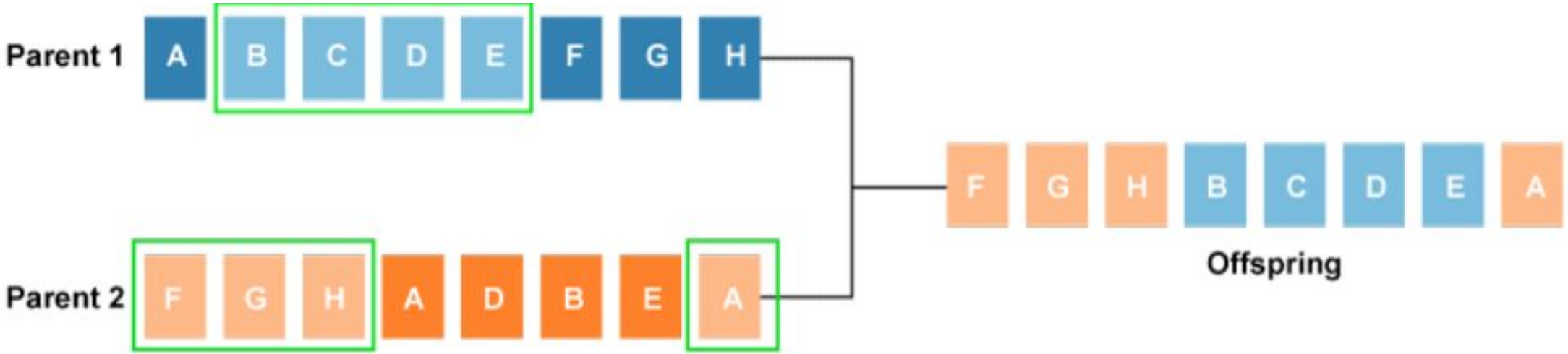
```
GA()
  initialize population

  find fitness of population

  while (termination criteria is reached) do
    parent selection
    crossover with probability pc
    mutation with probability pm
    decode and fitness calculation
    survivor selection
    find best
  return best
```

Crossover: The crossover plays a most significant role in the reproduction phase of the genetic algorithm. In this process, a crossover point is selected at random within the genes. Then the crossover operator swaps genetic information of two parents from the current generation to produce a new individual representing the offspring.

The genes of parents are exchanged among themselves until the crossover point is met. These newly generated offspring are added to the population. This process is also called or crossover.



Knapsack Problem by using Genetic Algorithm

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

There are four items. Each item is associated with some weight (W) and value at item (v).

There is a knapsack (k) with limited capacity that can hold atmost 12 kg.

Knapsack Problem by using Genetic Algorithm

STEP -1: Chromosomes Encoding

C_i

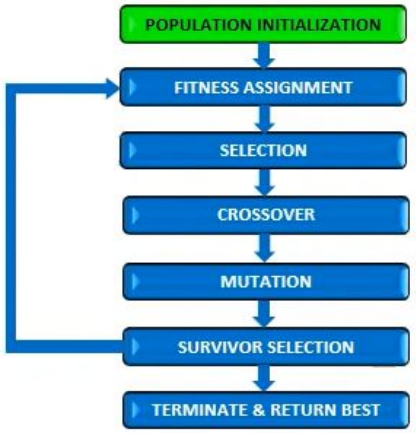
A	B	C	D

Gene: **0 –** represents absence of item in the knapsack
1 – represents presence of item in the knapsack

4 bits are requested to represent chromosomes encoding

Set space = 2^4

Initial population is created and chromosomes randomly created



ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Knapsack Problem by using Genetic Algorithm

STEP -2: Fitness Function

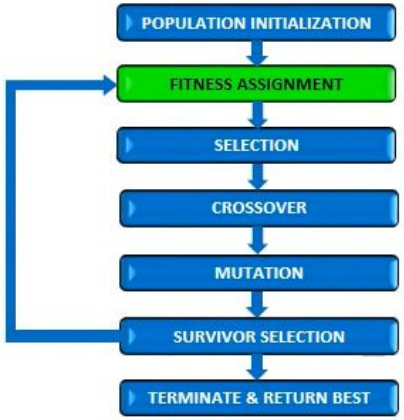
Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_1

C_1	0	1	1	0
	A	B	C	D

represent that knapsack has presence of item B & C and absence of item A & D

- value of knapsack = value of B + value of C
  = 5 + 10 = 15
- Weight of knapsack = weight of item B + weight of item C
  = 3 + 7 = 10 kg

Knapsack capacity = 12 kg as 12kg> 10kg **so C_1 is accepted**

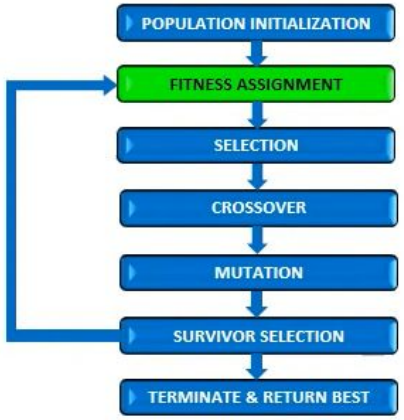


ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -2: Fitness Function

Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_2

C_2	0	1	0	1
	A	B	C	D

represent that knapsack has presence of item B & D and absence of item A & C

- value of knapsack = value of B + value of D
  = 5 + 7 = 12
- Weight of knapsack = weight of item B + weight of item D
  = 3 + 2 = 5 kg

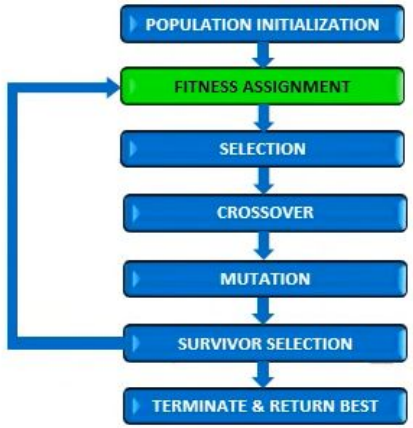
Knapsack capacity = 12 kg as 12kg> 5kg **so C_2 is accepted**

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -2: Fitness Function

Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_3

C_3	1	1	0	1
	A	B	C	D

represent that knapsack has presence of item A, B & D and absence of item C.

- value of knapsack = value of A + value of B + value of D
  = 12 + 5 + 10 = 15
- Weight of knapsack = weight of B + weight of C + weight of D
  = 5 + 3 + 7 = 10 kg

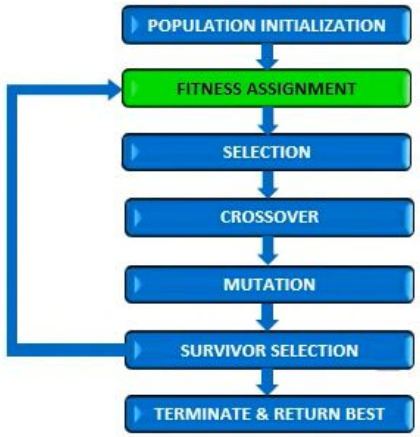
Knapsack capacity = 12 kg as 12kg> 10kg **so C_3 is accepted**

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -2: Fitness Function

Next Step is to determine fitness function which is used to evaluate how good particular solution is. Lets take C_4

C_4	1	1	1	1
	A	B	C	D

represent that knapsack has presence of all item A, B, C & D

- value of knapsack = value of A + value of B + value of C + value of D
  = 12 + 5 + 10 + 7 = 34
- Weight of knapsack = weight of B + weight of C + weight of B + weight of C
  = 5 + 3 + 7 + 2 = 17 kg

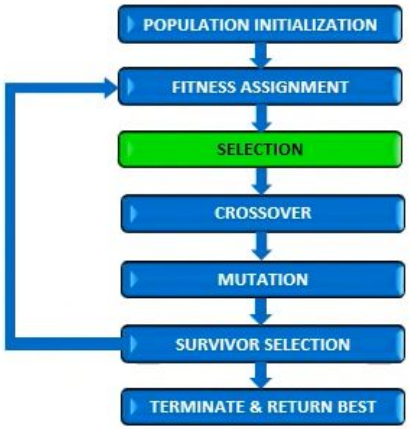
As 17 kg > knapsack capacity; 17 kg > 12 kg **so C_4 is discarded.**

ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Chromosome Encoding

C1	0	1	1	0
C2	0	1	0	1
C3	1	1	0	1
C4	1	1	1	1

Knapsack Problem by using Genetic Algorithm



STEP -3: Selection

Next step is to collect the filter individual and wake up the next generation, chromosome

- **By using Roulette Wheel Selection**

  - Spin the Roulette Wheel and whenever the wheel stops, the individual gets selected at that point.
  - The individual that has the highest fitness value gets larger share of the wheel

e.g. total fitness value = 15 + 12 +24 + 0 = 51

Fitness value of $\mathbf{C_3}$ = 24 ; largest fitness

So, $\mathbf{C_3}$ occupies half of the wheel as 24/51

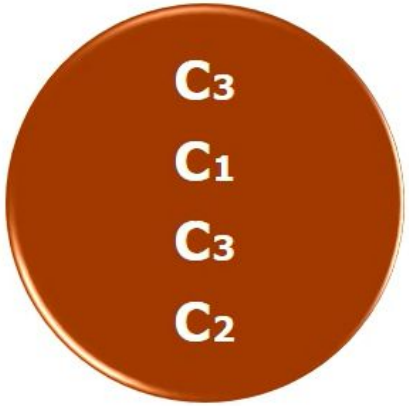
$\mathbf{C_4}$ has zero chance of wining, $\mathbf{C_3}$ has the highest probability of getting selected in the next generation

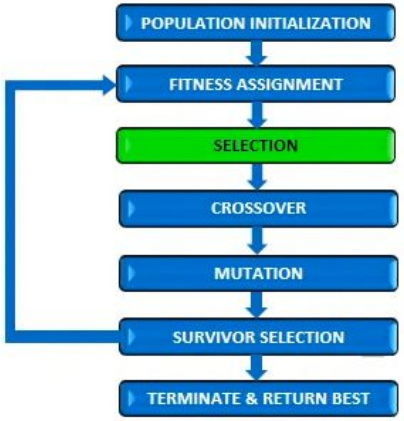
Knapsack Problem by using Genetic Algorithm

After spinning Roulette wheel, in the first spin **C_3** will be selected and then **C_1** after that **C_3** and **C_2**.

Individuals of next generation are selected as follows:

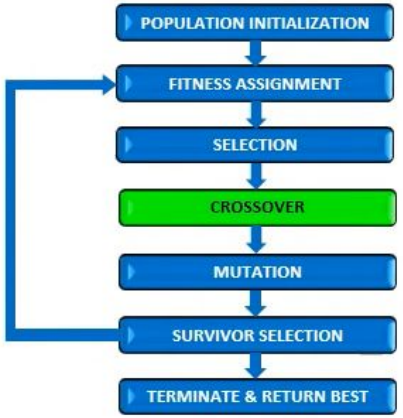
Generation 2





ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Knapsack Problem by using Genetic Algorithm



STEP -4: Crossover

The crossover operation takes the selected chromosomes for mating and mixes the genetic material to produce offspring.

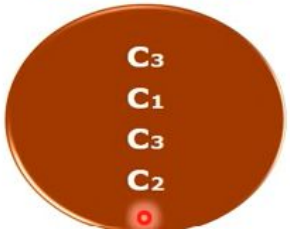
C_3	1	1	0	1
C_2	0	1	1	0
	A	B	C	D

One point crossover – Randomly select the position on the chromosomes about which gene would be exchange.

C_3	1	1	0	1
C_2	0	1	1	0

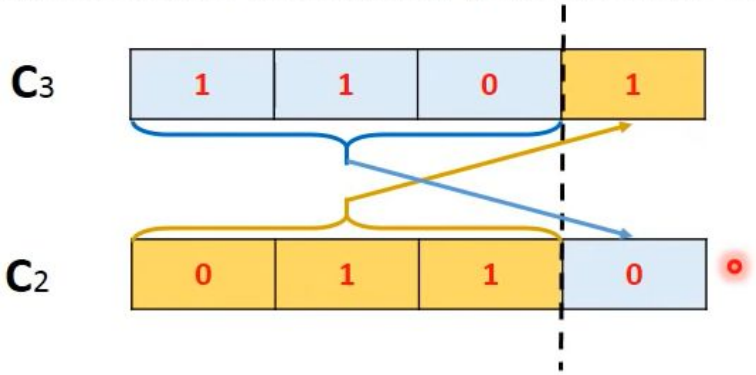
ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Generation 2

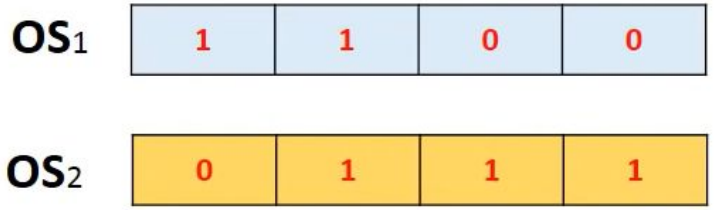


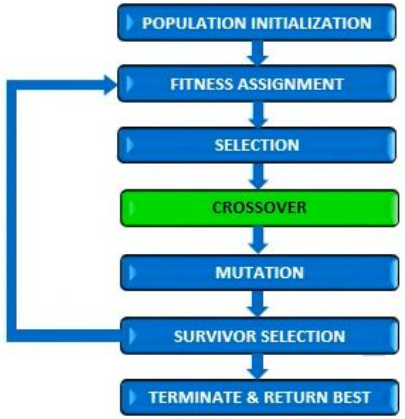
Knapsack Problem by using Genetic Algorithm

One point crossover – Randomly select the position on the chromosomes about which gene would be exchange.



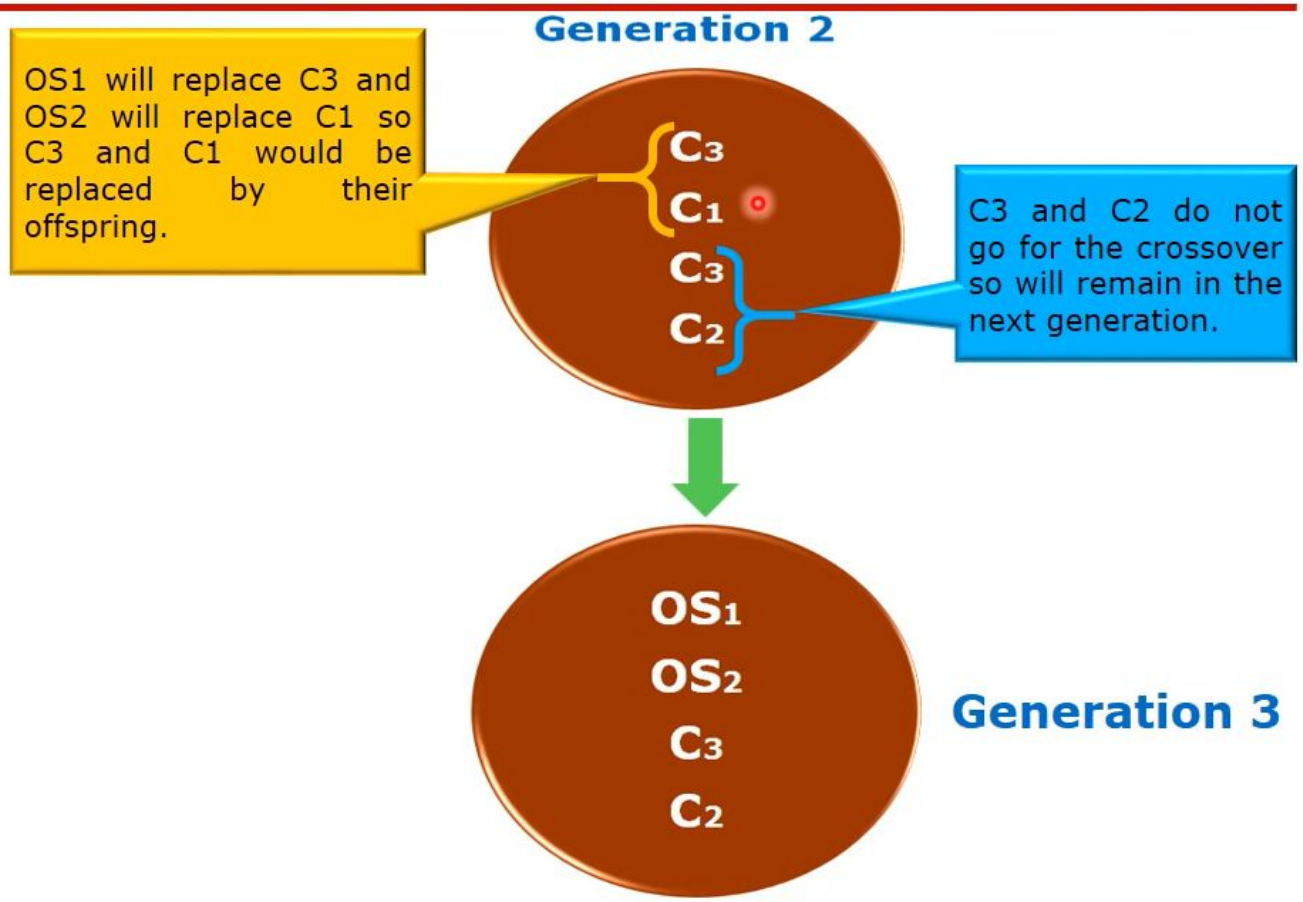
Result of one point crossover i.e. produced offspring

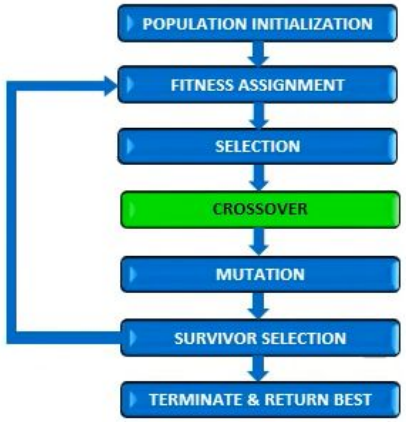




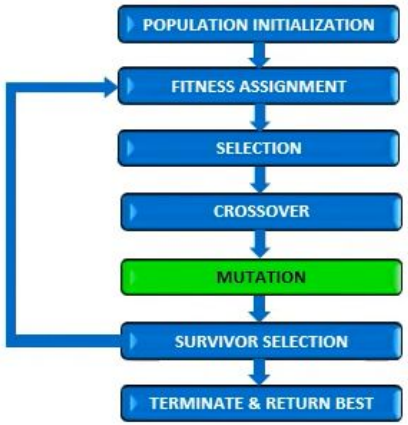
ITEM	WEIGHT	VALUE
A	5 kg	$12
B	3 kg	$ 5
C	7 kg	$ 10
D	2 kg	$ 7

Knapsack Problem by using Genetic Algorithm





Knapsack Problem by using Genetic Algorithm



STEP -4: Mutation

Introduces the diversity within the population so that search algorithm does not necessarily get stuck at local maxima.

Lets us consider chromosomes C. Now randomly select a gene from C**.** A Flip happens at the selected genes and zero become one and one becomes zero

C	0	1	1	0
C'	0	1	0	0